

Лекция 9

Изключения

```
throw <обект>
{
}
catch (тип)
```

```
main      f()      g()
f();      A obj;   B obj;
catch(int) g();   throw 37;
          X        X
```

Diagram showing stack unwinding: An arrow labeled ~A points from the throw statement in g() back to the catch statement in main. Another arrow labeled ~B points from the throw statement in g() back to the call to g() in f(). Both destinations have an 'X' below them, indicating destruction.

• stack unwinding - ако има catch-ване



Изключения в деструктора

- НЕ хвърляме изключения в деструктора
2 изключения → terminate



Изключения в конструктора

```
A() {
    throw ...;
}
```

не се извиква деструктора на A
(внимаваме за memory leak)

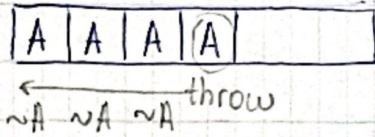
```
X() {
    str = new char[37]; → memory leak
    :
    throw 37;
}
~X() {
    delete [] str;
}
```

```
X {
    A obj1;
    B obj2;
    C obj3;
}
```

```
X(): ~A obj1(...), throw obj2(...), obj3(...)
```

единствено деструктора на A

`A* ptr = new A[10];`



⚠ Може да throw-не при някой елемент (например чрез static променлива, която се изменя)

У
`char* str1;`
`char* str2;`
`char* str3;`

У()
`str1 = new char[30];`
`try {`
`str2 = new char[...];`
`}`
`catch (std::bad_alloc &)`
`{ delete[] str1;`
`throw;`
`}`
`try`
`str3 = new...`
`catch`
`delete[] str1;`
`delete[] str2;`
`throw;`

или:

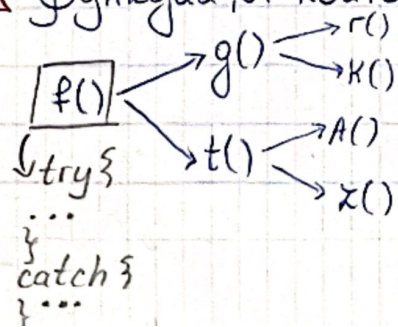
`char* arr = new (std::nothrow)`
`char[30];`

`if (!arr)`
`{`
`//... //`
`throw`
`}`

↓ Ако не успее
`→ nullptr`

(по-удобен начин)

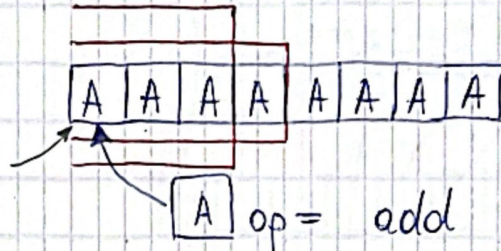
⚠ Функция, от която не хвърляме изключение



Колекции от обекти

→ клас, който съдържа колекция от еднотипни обекти

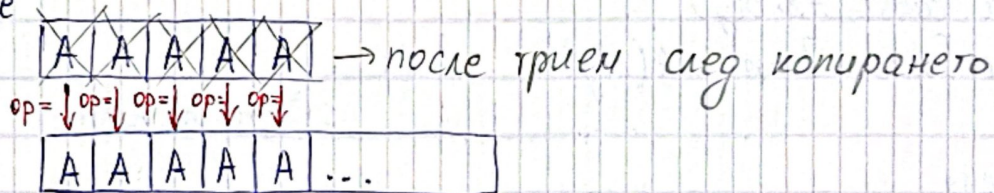
```
A* arr = new A[8];  
size = 0;  
cap = 8;
```



remove

```
(pop-back())  
arr[size-1].~A();  
size--;
```

resize



Проблеми: 1) на A му трябва default-ен конструктор
→ може да се избегне, ако задела паметта
предварително и върху нея извикваме
конструкторите

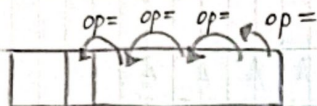
`A(5) A(9,7)` → placement new - приема вече заделена
памет и извиква конструктор върху нея

2) При resize: def. constr
↳ винаги се + op =
създават нови обекти

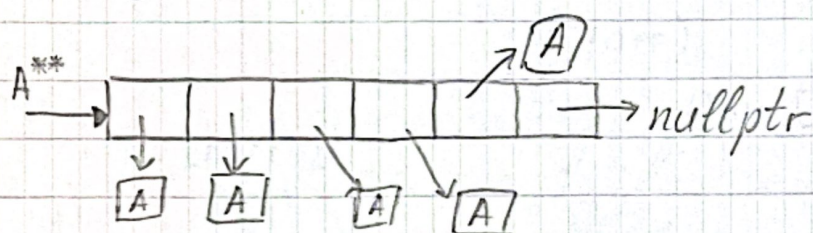
- пак може да се избегне с placement new

3) бавни размествания
3x op= за размяна

4) нахалне на елемент на индекс



Колекция от указатели към A



```
add(const A& obj)
arr[size++] = new A(obj); // KK
```

триене

A*

```
delete[] arr;
```

A**

for

```
delete arr[i];
```

```
delete[] arr;
```

Плюсове:

- 1) Не трябва default-ен конструктор на A
- 2) позволяват се празни позиции (nullptr)
- 3) Бързи размени
- 4) resize не създава нови обекти от A

A** newData = new A*[cap*2] { nullptr };

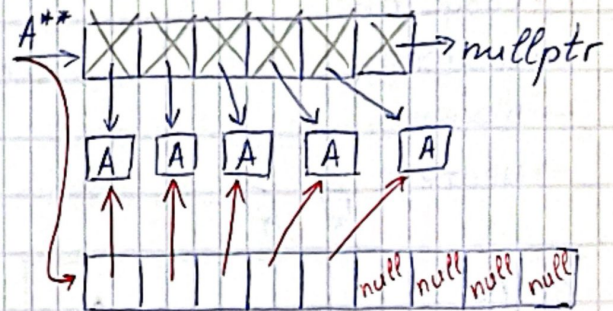
for ; to size {

newData[i] = data[i];

}

delete [] data;

data = newData;



copy From

for

{ if (data[i] != nullptr)

заделяне

else

- data[i] = nullptr;

}

A* се използва повече

A | A | A | A | A

→ locality → възползваме се, че обектите са на съседни адреси

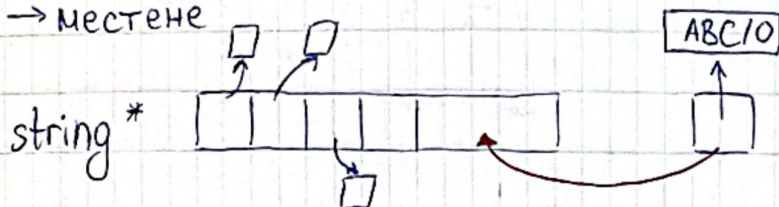
A** - "празни места"

"бързи шар-ове"

(при такива условия се иска A**)

Move семантики

- създаване
- копиране
- триене
- местене



```
add(const string& str)
data[i] = str;
```

Типове стойности в C++:

→ lvalue `int a = 7;`
 `a;`

- Име на съществуваща променлива / функция
- Извикване на функция, която връща референция

→ rvalue (частен случай на lvalue)

- литерали: `73`, `nullptr`, `"ABC"`, `false`
- извикване на функция, която връща копие

→ xvalue → `expiring value`

- обекти към края на жизнения си цикъл

```
A f() {  
    ...  
}
```

```
f().num++;
```

⚠ rvalue = `rvalue` + `xvalue`

`f(int& a)` — приема `lvalue`

`f(3)` $\times$

`f(const int&)` → `lvalue` / `rvalue`

def. rvalue reference
int &&

$f(\text{int} \&\& n)$ — приема само стойности

$f(3)$ ✓ отаква подадената променлива/константа да не

$f(n)$ ✗ се използва след функцията

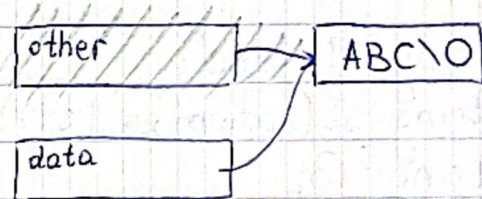
$\text{add}(\text{string} \&\& \text{str})$

↓

тук мога да открадна данните на str, а не да ги копирам.

→ конструктор за "открадване" на данните —
move конструктор

```
string  
char* data;  
string(string && other){  
    data = other.data;  
    other.data = nullptr;  
}
```



move op=

```
string & op = (string && other)  
if (this != &other)  
{ free();
```

```
    data = other.data;  
    other.data = nullptr;  
}
```

moveFrom

$f(A \&\& r)$

A obj;
 $f(obj);$ ✗

$f(A(3,4))$
 $\rightarrow A(int, int) \dots \sim A()$

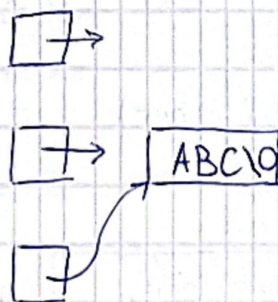
```
g(string && str) { ... }
string str;
g(str);
```

`std::move`
 $\rightarrow$ преобразува lvalue в rvalue

• декларираме, че от подаденото lvalue можем да крадем, защото няма да се използва след функцията

```
{
string s1="ABC";
string s2(std::move(s1)); // move конструктор
string s3;
s3 = std::move(s2); // move op=
}
```

3x $\sim$ string



```
f(string ("ABC")) // конструктор, деструктор
f(str1); // копиращ конструктор, деструктор
f(std::move(str2)); // move конструктор, деструктор
```

бпрос за теория:

```
g(A&& obj) {
  ...
}
```

```
{ A obj; // A()
  g(std::move(obj)); // нищо,
                      референция
                      към gvalue
}
```

③ $\sim A()$

```
{(A&& obj)
```

obj се третира
като lvalue

```
g(std::move(obj));
```

пр. в

```
g(A&& ...) {
  ...
}
```

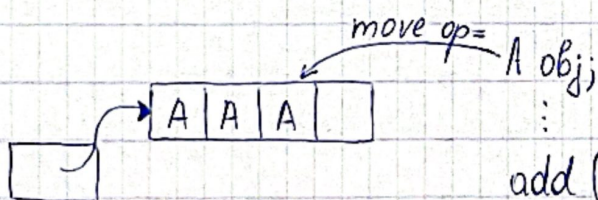
move конструктор
moveFrom(std::move(other));

```
add(const A& obj) {
```

```
  data[i] = obj; // копие
```

```
add(A&& obj)
```

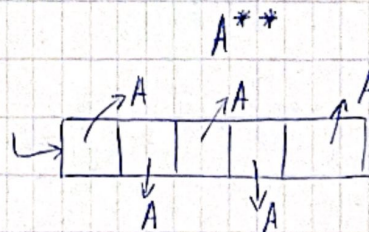
```
  data[i] = std::move(obj)
```



```
add(A&& obj) {
```

```
  data[i] = new A(std::move(obj));
```

```
}
```



- крадем целия масив от указатели

Big 4 + move конструктор + move op=

Big 6
(голямата шестуца)

$A(A \&\& \dots)$
 $A \& op= (A \&\& other)$ } прехсерт (няма задейване на памет или логика)

⚠️ нива на сигурност при изключения
(на семинар)